MYHOURLY

Backend Requirements and Planning

Single-organisation HRMS backend — requirements, data model, APIs and workflows

Document information	
Document type	Backend project requirements and planning
Purpose	Explain what the backend does, how its modules work together, how its database is structured, and how its APIs support the business workflows.
Document status	Project requirements and planning baseline. Requirements describe the intended system behavior, module responsibilities, data relationships, API expectations, and business workflows. Items that require a product or business decision are recorded as open decisions.

How to read this document Amber call-outs mark **open decisions** and items to **verify** against the source code before they are treated as final policy. Blue call-outs are general notes.

Contents

Section	Title	Covers
1	Project overview	Purpose, backend responsibilities, technologies used
2	Functional requirements	Roles, module inventory, module requirements, business rules
3	System architecture	Layering, request processing, dependencies
4	Database design	Migrations, tables and fields, indexes
5	Entity relationship diagram	Core entity relationships
6	API specifications	Common contract, endpoint register
7	Business workflows	Per-module workflows and cross-module flows
8	Security requirements	Spring Security / JWT configuration
9	Implementation roadmap	Phases, deliverables, acceptance criteria
10	Detailed module guide	Objective, data, rules and open points per module
11	Database relationship reference	Parent/child relationship table
12	API contract standards	Record to capture for every endpoint
13	How modules share data	Data flow, settings influence, traceability

1. Project overview

1.1 Purpose, problem, and objectives

Aspect	Description
What it is	MyHourly is a single-organisation HRMS backend that manages employee, attendance, leave, compensation, and HR-administration records through role-controlled APIs.
Capabilities supported	• Employee self-service • Manager approval • HR administration • Payroll preparation • Announcements and reporting • Performance review • Indian Form 16 / Form 16A tax-record maintenance
Planned backend scope	Authentication, security, employee management, organisation masters, attendance, leave, payroll, holidays, calendar, notifications, performance, reports, settings, lookups, Form 16, and Form 16A.
Out of scope	Recruitment, candidate management, project/timesheet management, payment-gateway processing, and multi-tenancy — unless separately approved.

1.2 Backend responsibilities

#	Responsibility
1	Authenticate users and enforce roles through Spring Security / JWT.
2	Persist and validate HR domain records using JPA / PostgreSQL.
3	Execute attendance, leave, payroll and approval rules transactionally.
4	Expose REST APIs, common success/error payloads, pagination, reports and PDF/file responses.
5	Send mail, store announcement attachments through an S3-compatible B2 service, and run scheduled HR reminders/maintenance.

1.3 Technologies used

Category	Technology	Used for
Language / runtime	Java 21	Backend implementation language and runtime.
Application framework	Spring Boot 4.0.7	Application bootstrap, configuration and dependency management.
Web / REST	Spring MVC	REST controllers, request binding, multipart handling and response mapping.
Security	Spring Security	Stateless authentication, role-based access control and method-level `@PreAuthorize` guards.
Token handling	JJWT	Creating and validating JWT access and refresh tokens.

Category	Technology	Used for
Validation	Spring Validation (Bean Validation)	Request DTO validation with `@Valid`.
Persistence	Spring Data JPA	Entity mapping and repositories (derived queries, JPQL, specifications).
Database	PostgreSQL	Primary relational data store (active profile).
Schema migration	Flyway	Versioned database migrations (V1–V11 and later).
Email	Spring Mail	Password-reset and notification email.
File / object storage	AWS SDK — S3-compatible B2 client (Backblaze B2)	Storing announcement attachments.
Spreadsheet export	Apache POI (XLSX)	Excel report exports.
PDF generation	OpenPDF	Payslip and other PDF output.
API documentation	OpenAPI / Springdoc	Generated API documentation and Swagger UI.
Developer productivity	Lombok	Boilerplate reduction in entities and DTOs.
Build	Maven (`pom.xml`)	Dependency management and build.

Note `pom.xml` also retains a MySQL runtime driver, but the active profile properties use PostgreSQL.

2. Functional requirements

2.1 Roles

Role (RoleName)	Notes
`SUPER_ADMIN`	Highest-privilege administrative role.
`HR_ADMIN`	HR administration role.
`MANAGER`	Manager approval role; scoped by the reporting hierarchy.
`EMPLOYEE`	Employee self-service role.
`PAYROLL_ADMIN`	Used by the payroll APIs.
`CLIENT`	No module-specific endpoint guard was inspected.

Authoritative source The controller method annotations (`@PreAuthorize`) are the authoritative source of endpoint permissions.

2.2 Backend module inventory

The table below is the official module inventory for this document, with related features listed as submodules.

No.	Backend module	Included features / submodules
1	Authentication	Login, token refresh, logout, password change/reset, current-user profile
2	User and Admin Management	User registration, role assignment, account status, administrative operations
3	Organization and Master Data	Departments, designations, job titles
4	Employee Management	Employee profiles, employee codes, profile photos, reporting-manager relationships
5	Lookup Services	Department/designation/job-title lookups, reporting-manager lists, employee ID/name lists
6	Attendance Management	Check-in, breaks, checkout, daily attendance, history, monthly summaries, calendar, dashboard
7	Attendance Regularization	Correction requests, regularization details, review actions, reversals
8	Leave Management	Leave types, leave requests, cancellation endpoint where supported, approval history
9	Leave Balance and Allocation	Leave balances, transactions, annual allocation, monthly paid/LOP allocation
10	Leave Policy Settings	Leave allocation rules, carry-forward configuration, sandwich-leave policies
11	Holiday and Calendar Management	Holidays, upcoming holidays, calendar data, attendance eligibility settings
12	General Settings	Attendance settings, leave settings, company settings
13	Payroll Management	Payroll generation, recalculation, status lifecycle, payroll history
14	Salary Management	Salary templates, salary structures, revisions, employee payment details
15	Payslip Management	Payslip retrieval and PDF generation
16	Tax and Form 16 Management	Form 16, employer master, salary, exemptions, deductions, verification, quarters and challans
17	Form 16A Management	Form 16A-related records and operations
18	PF and ESI	Provident fund and employee state insurance information and related payroll details, where supported
19	Project and Timesheet Management	Project records, timesheet submission and approval, including client approval where applicable
20	Notifications and Announcements	User notifications, unread counts, announcements, notification read status
21	Celebration Wall	Today's celebrations, upcoming birthdays, celebration data
22	Reports and Analytics	Attendance reports, leave reports, report filters and exports where supported

No.	Backend module	Included features / submodules
23	Performance Management	Performance reviews, summaries, completion, search and review history

Verify Some entries above are grouped responsibilities rather than independently verified controller modules. In particular, Project / Timesheet Management and PF / ESI should be confirmed against the actual backend API and source code before being marked as implemented. (Section 1.1 lists project/timesheet management as out of scope unless separately approved.)

Detailed capability inventory

The same capabilities listed at a finer grain (32 items). Several are grouped into shared backend modules in the table above.

No.	Module	Main responsibilities
1	Authentication and Authorization	Login, authentication, JWT handling, role-based access control (RBAC), and access restrictions
2	User and Admin Management	User accounts, role assignment, account administration, and administrative access
3	Organization Management	Manage organizational information and company structure
4	Department Management	Create and manage departments
5	Designation and Job Title Management	Maintain designations and job titles
6	Employee Management	Employee records, employee codes, reporting relationships, and employment information
7	Employee Directory	View and search employee information according to permissions
8	Attendance Management	Check-in, checkout, daily attendance records, working-hour calculations, and attendance status
9	Attendance Regularization	Submit, review, approve, or reject attendance correction requests
10	Shift Management	Configure shifts and working schedules
11	Holiday Calendar Management	Maintain holidays and designated non-working days
12	Leave Management	Apply for leave, review requests, and manage leave statuses
13	Leave Policy and Settings	Configure leave rules, monthly paid-leave limits, carry-forward behavior, and sandwich-leave rules
14	Leave Balance and Paid/LOP Management	Maintain leave balances and determine paid leave versus Loss of Pay (LOP)
15	Project Management	Manage projects and associated project information
16	Timesheet Management	Record employee project hours and manage timesheet submissions
17	Timesheet Approval	Review and approve or reject submitted timesheets, including client approval where applicable
18	Payroll Management	Prepare payroll, calculate payroll amounts, and manage payroll lifecycle statuses
19	Salary Template Management	Define reusable salary templates

No.	Module	Main responsibilities
20	Salary Structure Management	Maintain employee salary components and salary structures
21	Payslip Management	Generate and provide employee payslips in PDF format
22	Tax Center	Manage tax-related employee information and tax documents
23	Form 16 Management	Generate or manage Form 16 documents
24	Form 16A Management	Manage Form 16A documents where applicable
25	PF and ESI Management	Maintain provident fund and employee state insurance information and related payroll details
26	Performance Management	Manage employee performance reviews and related information
27	Notifications	Deliver system notifications and updates
28	Announcements	Publish and manage company announcements
29	Celebration Wall	Display employee celebrations and relevant company events
30	Reports and Analytics	Provide operational reports and summaries across HRMS modules
31	General Settings	Manage configurable system and company settings
32	File and Document Storage Integration	Support storage of generated or uploaded files where required by modules

2.3 Modules and requirements

ID	Module / functional requirement	Backend responsibility / expected behavior
FR-AUTH	Authenticate with credentials; refresh/logout; change/reset password; return current profile.	AuthenticationController, AuthenticationServiceImpl, refresh/revoked/reset-token entities.
FR-ADM	Privileged users register users, assign role, change account status, create/update employee profile and review attendance.	AdminController, AdminServiceImpl.
FR-ORG	Maintain department, designation and job-title hierarchy and employee/reporting-manager profile data.	master / employee / lookup packages.
FR-ATT	Employees record check-in, break start/end and checkout; system reports today/history/month/calendar/dashboard.	AttendanceServiceImpl, AttendanceSettings.
FR-REG	Authorised employee submits multi-day attendance correction; manager/HR/super action/revert its details.	AttendanceRegularizationServiceImpl.
FR-LEV	Maintain active leave types/balances; apply/cancel/approve leave, ledger transactions, annual allocation and approval history.	Leave services / controllers.
FR-PAY	Maintain templates, salary structures/revisions, payment details, payroll versions/status/history and payslip PDF.	Payroll services / controllers / PDF classes.
FR-HR	Maintain holidays, global calendar, company/attendance/leave settings, notifications, announcements, celebrations and reports.	holiday / calendar / settings / notification / report packages.

ID	Module / functional requirement	Backend responsibility / expected behavior
FR-PERF	Create/update/complete/search/delete performance reviews and review summaries.	PerformanceServiceImpl.
FR-TAX	Maintain Form 16, employer master, salary, exemptions, deductions, Chapter VIA, verification, quarter and challan records.	form_16/**, Form_16A/**.

2.4 Business rules and constraints

Area	Rule	Reference
Employee structure	An employee belongs to one department, designation, job title and user account; reporting manager is an optional self-reference.	Employee entity
Uniqueness	Department/designation/job-title codes, company code, holiday date, leave type name, payroll number, refresh token and reset-token hash have database uniqueness as defined by V1. Employee code is made unique by V3.	V1, V3
Attendance status	Allowed values: `PRESENT`, `LATE`, `HALF_DAY`, `ABSENT`, `LEAVE`, `HOLIDAY`, `WEEKEND`, `MISSED_CHECKOUT`. Settings control office times, grace, minimum/half-day minutes, overtime and weekend/holiday attendance.	AttendanceSettings
Leave validation	Verifies active type, date ordering and overlapping pending/approved applications. Normal working-day calculation excludes weekends/holidays; configured Friday/Monday sandwich policies can force weekend dates into the leave calculation.	LeaveValidationServiceImpl, V11
Approval authority	A manager approval is checked against the employee reporting hierarchy. Approval produces a trail; approved paid days affect balance/transactions, while LOP does not deduct annual balance.	LeaveAuthorizationServiceImpl
Paid vs LOP	Paid leave and Loss of Pay (LOP) days are tracked separately and attributed to the relevant calendar month. Legacy records without allocation details require a defined fallback rule.	V9, V10
Payroll	Rejects another active payroll for the same employee/month; gross, LOP, half-day amount, deductions and net payable are recalculated. Status values: `DRAFT`, `GENERATED`, `APPROVED`, `PAID`, `SUPERSEDED`, `CANCELLED`.	PayrollServiceImpl

3. System architecture

3.1 Design and request processing

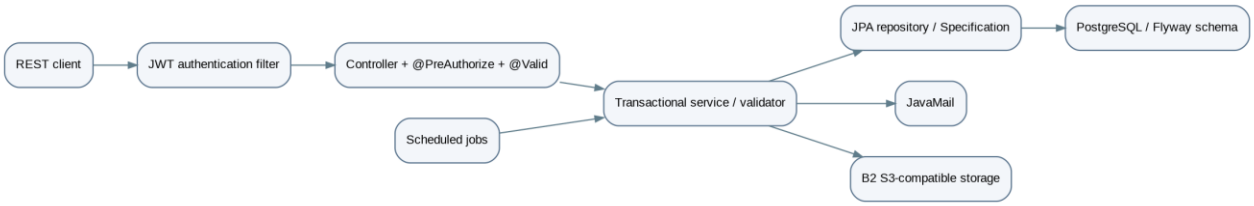


Figure 1. Layered request-processing flow

Layer	Responsibility
Controllers	Bind path/query/body/multipart input, use `@Valid` where declared, call service interfaces, and return either `ApiResponse<T>` / `PageResponse<T>` or direct DTO/binary responses.
Services	Implement business transactions, mapping and authorisation.
Repositories	Provide derived / JPQL / specification-based persistence.
Entities	Map domain state.
Exception handling	GlobalExceptionHandler, ErrorResponseFactory, ApiError, and custom bad-request / business / duplicate / forbidden / not-found / unauthorised exceptions centralise failure representation.

3.2 Major dependencies

Source / prerequisite	Dependency relationship
Organisation data	Prerequisite for employee records.
Employee	Prerequisite for attendance, leave, payroll, performance, notifications and Form 16.
Holiday and settings	Influence attendance and leave calculations.
Approved leave and attendance	Feed payroll calculations.
Notification / email	An effect of leave, attendance and announcement processes.
Reports	Query attendance and leave without becoming an authoritative source.

4. Database design

This section describes the tables and constraints used to store backend data. Database schema changes must remain consistent with the current entity mappings and constraints.

4.1 Schema evolution and data model

The database design includes the baseline schema and subsequent schema changes. The final implementation must keep entity mappings, constraints, indexes, and database migrations aligned.

Migration	Change
V1	Baseline schema (`db/migration/V1__baseline_schema.sql`).
V2	Payroll joining / template constraint.
V3	Unique employee code.
V4	Employee-type text.
V5	Form 16 suite.
V6	Payroll half-day fields.
V7	Removes salary-template employee-type check / unique restriction.
V8	Adds query indexes.
V9	Paid / LOP / approver fields on leave requests.
V10	Monthly leave allocations.
V11	Sandwich-policy fields and forced-date JSON.

4.2 Tables and exact field design

Convention	Meaning
Identifiers	All baseline IDs are `bigint` generated by default as identity.
Audit fields	`created_at timestamp(6) NOT NULL`, `updated_at timestamp(6)` unless noted.
PK / FK	PK denotes primary key; FK references the listed relationships.
Marker !	Fields marked ! are NOT NULL.
Exact DDL	Exact V1 DDL: `src/main/resources/db/migration/V1__baseline_schema.sql`.

Table	Columns (type; ! non-null)
`users`	id PK, created_at!, updated_at, last_login, password_changed_at timestamps; username varchar(50)!, email varchar(100)!, password varchar(255)!; role varchar(30)! check RoleName; user_status varchar(30)! check ACTIVE/INACTIVE/LOCKED/DISABLED/PASSWORD_EXPIRED.
`employees`	id PK; user_id bigint! unique FK users; department_id, designation_id, job_title_id bigint! FKs; reporting_manager_id bigint self FK; names varchar(50); email varchar(255)! unique, phone 15; employee code unique after V3; dates; profile photo OID/name/type; active boolean!; enums employment type, gender, role.
`departments`	id/audit, department_code varchar(20)! unique, department_name varchar(100)!, description varchar(255), active boolean!.
`designations`	id/audit, department_id bigint! FK, code/name varchar(255)! unique, description, active.
`job_titles`	id/audit, designation_id bigint! FK, job_title varchar(100)!, code varchar(255)! unique, active.

Table	Columns (type; ! non-null)
attendance	id/audit; employee_id bigint! FK; attendance_date date!; check-in/out timestamps and numeric(10,7) locations; working/break/late/early/overtime minutes; status and employee-state enum checks.
attendance_breaks	id, attendance_id bigint! FK, start/end timestamps, break_minutes integer, break_type varchar(255)! check.
attendance_regularization	id/audit, employee FK, from/to dates!, request/approval/rejection dates, approver/rejector IDs, status varchar(30)! check, reason/rejection reason varchar(500).
attendance_regularization_detail	id/audit, regularization/attendance FKs, original/requested/approved in/out timestamps/statuses; status check PENDING/APPROVED/REJECTED/REVERTED; remarks varchar(500).
attendance_settings	id/audit; active, cutoff/grace/minimum/half-day integer!, office times!, overtime/weekend/holiday flags!.
holidays	id/audit; holiday_date date! unique, name varchar(100)!, description varchar(300), active/recurring/attendance_allowed booleans!, type enum check.
leave_types	id/audit; name varchar(50)! unique, description, allocated days/monthly guideline integers!, paid/carry-forward/active booleans!.
leave_settings	id/audit; annual paid leave/monthly guideline integers!, active/carry-forward booleans!; V11 sandwich Monday/Friday/Friday-Monday booleans! default false.
leave_balances	id/audit; employee/type FKs!, year/allocated/used/expired/remaining integers!; unique(employee, type, year).
leave_requests	id/audit; employee/type FKs!, dates!, total days!, status check, reason varchar(500)!; V9 paid_days, lop_days, approved_by FK employees; V11 forced_working_days_json text.
leave_approvals	id/audit; request and approver FKs!, action APPROVE/REJECT and level MANAGER/HR checks!, remarks varchar(500).
leave_transactions	id/audit; employee/type FKs!, optional request FK, before/after/days ints!, transaction-type check, remarks.
leave_request_month_allocations	V10: id/audit, request FK!, allocation month date!, paid/working days ints!, unique(request, month), cascade delete request.
salary_templates	id/audit; active; salary components numeric(12,2)!; employee_type varchar(30)!. V7 removes its original check and unique constraint.
salary_structures	id/audit; employee FK!, optional template FK, effective dates, status check ACTIVE/INACTIVE, components numeric(12,2)!, remarks.
employee_payment_details	id/audit, employee FK! unique; bank/account/IFSC required; optional PAN/ESI/PF/UAN; payment mode check.
payrolls	id/audit; employee/structure FKs!, approver FK; payroll/employee display fields, month/date/status/version, days, all component/deduction amounts numeric(12,2)!, payroll number varchar(30)! unique; V6 half_days int! default 0, half_days_amount decimal(12,2)! default 0.00.
payroll_history	id/audit, payroll FK!, optional performer FK, action check, remarks varchar(500).
performance_reviews	id/audit, employee/reviewer FKs, review period/type/status/rating/score, feedback text; unique employee/type/month/year.

Table	Columns (type; ! non-null)
`notifications`	id/audit, employee FK!, reference ID, read flag!, priority/type/reference-type checks, title varchar(150)!, message varchar(1000)!.
`announcements / announcement_attachment_urls`	announcement audit/id/title 150!/message 1000!/upload-type check; attachment join announcement_id FK, attachment_url varchar(1000).
`company_settings`	id/audit, active/working days, company code varchar(30)! unique, identity/contact/address fields and timezone/currency.
`security token tables`	• `refresh_tokens` — user FK, unique token, expiry/revocation. • `revoked_tokens` — unique token, expiry. • `password_reset_tokens` — user FK, unique hash, expiry/used timestamp.
`Form 16 suite`	Created/altered by V5; exact table/column DDL and FKs are in V5 because V1 intentionally does not define all Form 16 tables. Entity classes under form_16/entity and Form_16A/entity are the inspected mapping source.

4.3 Constraints, indexes, and differences

Item	Detail
V8 indexes	• attendance (employee, date) and (date) • attendance breaks — attendance FK • regularization — employee and detail FKs • reporting manager; designation / job-title lookup FKs • leave request — employee / status / update; leave transaction / request; approvals • notifications — employee / read / created • performance reviewer • payroll — employee / month / history • salary structure — employee / status • users — username / email • refresh token — user / revoked
Deliberate omission	V8 deliberately does **not** make username/email unique.
Schema differences	Entity/schema discrepancies are not silently inferred: • V7 explicitly relaxes template employee type. • V9–V11 augment baseline leave tables. • V5 is required for the Form 16 physical schema.

5. Entity relationship diagram (ERD)

The diagram below presents the core relationships between the main HRMS entities. Form 16 and Form 16A data relationships should be represented alongside the relevant tax-record entities in the final data model.

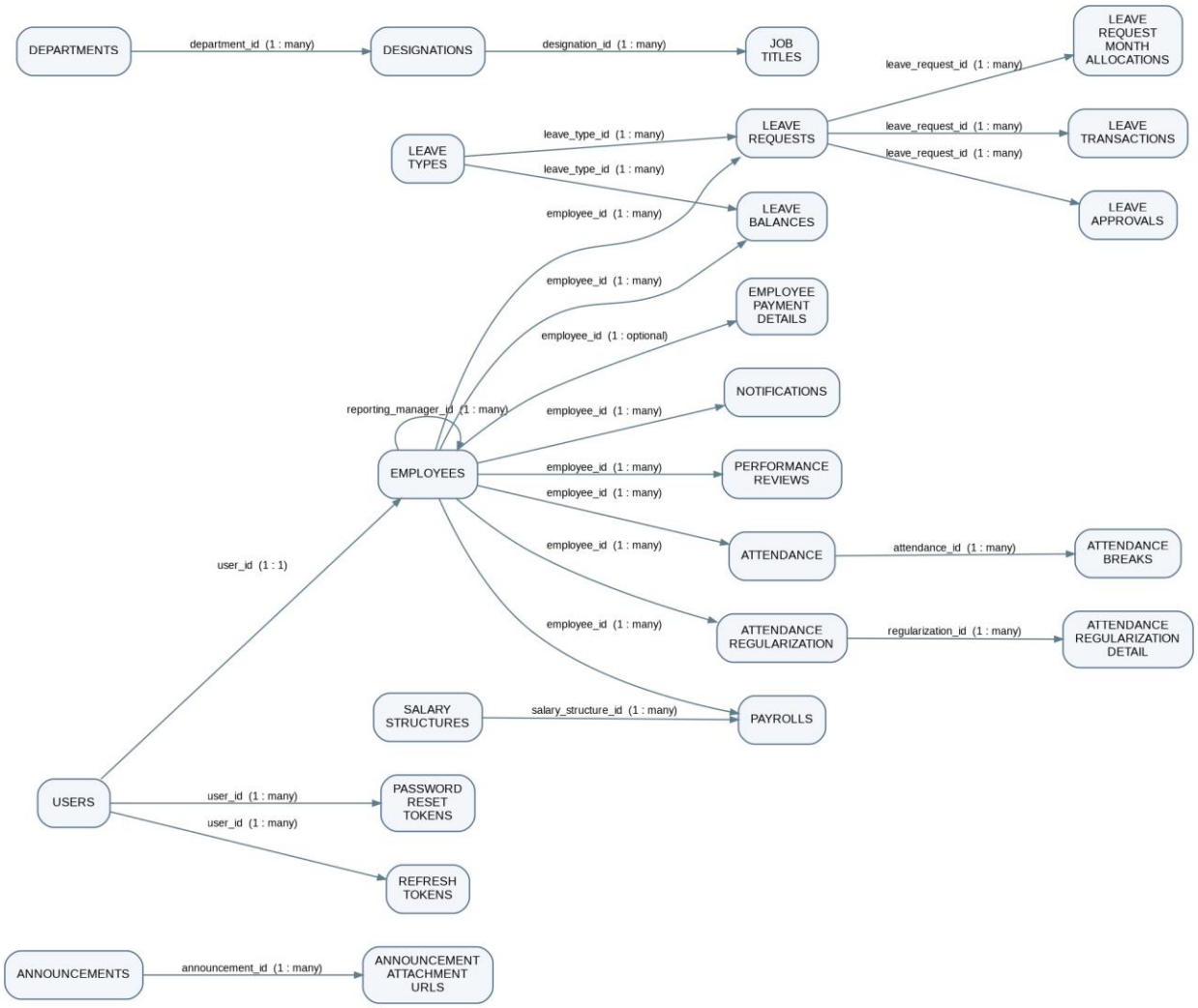


Figure 2. Core HRMS entity relationships

ERD note	Detail
Scope	The ERD intentionally excludes relationships not defined in V1 / V5 / entity mappings.
`announcement_attachment_urls`	A value-collection join table.
`employees.reporting_manager_id`	The self-referencing management hierarchy.
`Form 16`	Relationships are documented in V5, whose exhaustive table suite must be consulted for that additional subgraph.

6. API specifications

The endpoint list below is the module-level API register. Each endpoint specification should define its HTTP method, path, permissions, request fields, response structure, validation rules, and error behavior.

6.1 Common contract

Topic	Contract
Authentication	Protected calls use `Authorization: Bearer <JWT>`.
DTO location	Request DTOs are under each module's `api/request` or `dto/request`; response DTOs are adjacent.
Validation	`@Valid` triggers Bean Validation where the method declares it.
Response wrapper	`ApiResponse<T>` is common, but Form 16 / payroll endpoints often return direct DTOs, report endpoints return JSON or export bytes, and profile/payslip endpoints return bytes.
Status codes	Controller-defined (create typically 201, normal read/update 200).
Errors	Mapped by `GlobalExceptionHandler`.

6.2 Complete active endpoint register

Module / controller	Method	Complete URL	Purpose / input / service
Auth	POST	`/api/v1/auth/login, /token/refresh, /logout, /change-password, /forgot-password, /reset-password`	Respective request DTOs; AuthenticationService. Auth prefix is public at filter level; /me requires isAuthenticated.
	GET	`/api/v1/auth/me`	
Admin	POST	`/api/v1/admin/users/register, /employee-profile/{userId} (multipart), /users/{userId}/roles`	Class guard: super / manager / HR; AdminService.
	PUT	`/api/v1/admin/employee-profile/{userId}`	
	GET	`/api/v1/admin/users, /users/{userId}, /attendance, /attendance/employee/{employeeId}`	
	PATCH	`/api/v1/admin/users/{userId}/status`	
Employee	PUT	`/api/v1/employees/profile-photo (multipart), /api/v1/employees`	Employee service; broad self update, admin/manager listing.
	GET	`/api/v1/employees/{id}, /, /dropdown, /me, /{id}/profile-photo`	
	PATCH	`/api/v1/employees/{id}/status`	
Master (departments, designations, job-titles)	POST	`/api/v1/{resource}`	Create/update DTO per resource; HR/manager guards. Same routes for each of the three resources.
	GET	`/api/v1/{resource}/{id} and /`	
	PUT	`/api/v1/{resource}/{id}`	
	PATCH	`/api/v1/{resource}/{id}/status`	
	DELETE	`/api/v1/{resource}/{id}`	
Lookup / calendar / holiday	GET	`/api/v1/lookups/departments, /departments/{id}/designations, /designations/{id}/job-titles, /reporting-managers, /employee-id-name`	Lookup, calendar and holiday services; holiday mutation restricted to HR/manager.
	GET	`/api/v1/calendar`	
	POST / GET / PUT / DELETE	`/api/v1/holidays`	

Module / controller	Method	Complete URL	Purpose / input / service
	GET	/api/v1/holidays/calendar, /upcoming	
Attendance	POST	/api/v1/attendance/check-in, /break-start, /break-end, /check-out	Check-in/out/break DTOs; employee/manager/HR/super guard; AttendanceService.
	GET	/api/v1/attendance/today, /history, /monthly-summary, /calendar, /dashboard	
Regularisation	POST	/api/v1/attendance-regularizations	Create/action detail DTOs; manager/HR/super action.
	GET	/api/v1/attendance-regularizations/my, /pending, /all, /{id}	
	PATCH	/api/v1/attendance-regularizations/{regularizationId}/details/{detailId}	
	POST	/api/v1/attendance-regularizations/{regularizationId}/details/{detailId}/revert	
Leave	POST	/api/v1/leave-requests	LeaveRequestRequest / LeaveActionRequest; services listed in §7.3.
	PUT	/api/v1/leave-requests/{id}/cancel	
	GET	/api/v1/leave-requests/{id}, /my, /team, /	
	CRUD	/api/v1/leave-types	
	GET	/api/v1/leave-balances/my, /employee/{id}, /{id}, /	
	GET	/api/v1/leave-transactions/my, /employee/{id}, /leave-request/{id}	
Leave action / allocation	PUT	/api/v1/leave-approvals/{id}/leave-approval-by-manager	Manager-action DTO; allocation service.
	GET	/api/v1/leave-approvals/leave-request/{id}	
	POST	/api/v1/leave-allocation/all-employee, /leave-type/{id}	
Settings / notifications	GET / PUT	/api/v1/settings/attendance, /leave, /company	Settings DTOs; notification paging / announcement request.
	GET	/api/v1/notifications, /unread-count, /announcement/today, /upcoming-birthdays, /celebration-wall/today	
	PATCH	/api/v1/notifications/{id}/read, /read-all	
	POST	/api/v1/notifications/announcement (multipart)	
Payroll	Templates	/api/v1/payroll/salary-templates	Payroll DTOs; payroll roles as controller annotations.
	Structures	/api/v1/payroll/salary-structures, /revision, /{id}, /employee/{id}	
	Payment details	/api/v1/payroll/payment-details	
	POST	/api/v1/payroll/generate	
	GET	/api/v1/payroll/{id}, /number/{n}, /employee/{id}, /month, /status	
	PUT / PATCH	/api/v1/payroll/{id}	
	POST	/api/v1/payroll/{id}/regenerate	
	GET	/api/v1/payroll/{id}/payslip	

Module / controller	Method	Complete URL	Purpose / input / service
Reports / performance	GET	`/api/v1/reports/attendance, /leave`	Report filters/query; performance DTOs.
	POST	`/api/v1/performance, /search, /get-summary`	
	PUT	`/api/v1/performance/{id}, /{id}/complete`	
	GET / DELETE	`/api/v1/performance/{id}`	
Form 16	Various	`/api/v1/form16/{employeeId}, /{id}, /employee/{id}, activation / delete routes`	Separately guarded CRUD per tax subsection. See the note below.
	Various	`/api/form16/{form16Id}/{salary | exemption | …} (subsection routes)`	

Source note The Form 16 row was truncated in the supplied document (it ended at `/api/form16/{form16Id}/{salary` followed by `exemption`). Confirm the exact subsection routes against the Form 16 controllers.

The API register should document active endpoints only. For each endpoint, specify optional query parameters, request validation, response type, and applicable authorization rules.

7. Business workflows

These workflows describe the expected business paths from request initiation through validation, processing, persistence, and response. Calculation formulas and edge-case behavior should follow the approved business rules.

7.1 Authentication

Stage	Behavior
Login	Validates credentials through the authentication manager, loads custom user details, produces access/refresh tokens and records access state.
Refresh	Validates a non-revoked refresh token.
Logout	Revokes the refresh token.
Password reset	Creates/validates expiring hashed reset tokens and sends mail.
Failures	Authentication, validation, expired/revoked-token or account-status exceptions.

7.2 Attendance and regularisation

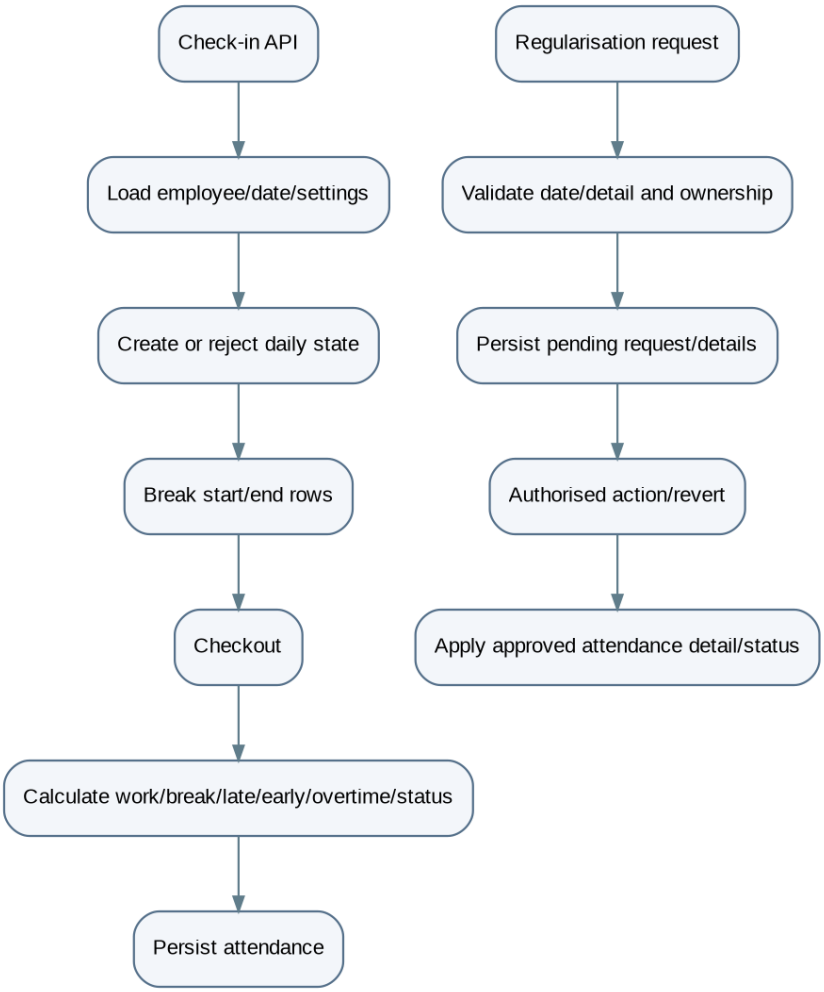


Figure 3. Attendance and regularisation overview

7.3 Leave



Figure 4. Leave processing overview

Aspect	Detail
Participating services	• LeaveValidationService • LeaveAuthorizationService • LeavePaidLopService • LeaveBalanceService • LeaveTransactionService • LeaveApprovalService • LeaveAllocationService • LeaveRequestService
Persistence	Database writes are to the tables specified in §4.
Failure behavior	Approval failure leaves no approved balance mutation because these operations are transactional.

7.4 Payroll

Stage	Behavior
Generation	Validates employee/month and absence of an active duplicate, reads the active salary structure and attendance summary, calculates gross/LOP/half-day/deductions/net payable, and writes payroll and history.
Update / regeneration	Recompute derived totals; status/paid events add history.
PDF generation	Validates payroll before emitting binary content.
Failure cases	• Missing employee / structure • Duplicate active payroll • Invalid calculated salary constraints

7.5 Other workflows

Module	Workflow
Masters / holidays / settings	Validated CRUD, with master hierarchy validation.
Notifications & announcements	Notifications are created/read/paged; announcements validate multipart data, upload B2 attachments, and persist announcement/URLs.
Reports	Filter attendance/leave and select JSON/Excel/PDF export.
Performance	Creates a period-specific review, updates/completes it and derives summaries.
Form 16	Create a form for an employee, then maintain each associated tax subsection with separately guarded CRUD.

7.6 Module-by-module workflow catalogue

This section defines the workflow for each backend module, including the normal processing path, key validations, and expected data changes. Any policy that has not been formally decided is listed as an open decision.

7.6.1 Authentication and account lifecycle

Main path: sign in

Step	Action
1	The client submits login credentials to the authentication API.
2	The authentication service delegates credential verification to Spring Security.
3	The service checks the account/user state as implemented.
4	On success, access and refresh tokens are created and token state is persisted where required.
5	The API returns the authentication response.
6	Protected requests later pass through the JWT filter before reaching controllers.

Refresh, logout, reset, profile

Path	Flow
Refresh	Receive refresh token → validate token, expiry, and revocation state → resolve user → issue the applicable token response.
Logout	Receive token/session context → revoke the refresh token or applicable token state → return the logout result.
Password reset	Accept reset request → create a hashed, expiring reset token → send reset message → validate token on submission → update password and mark token used.
Current profile	Authenticate caller → resolve current user → return profile data.

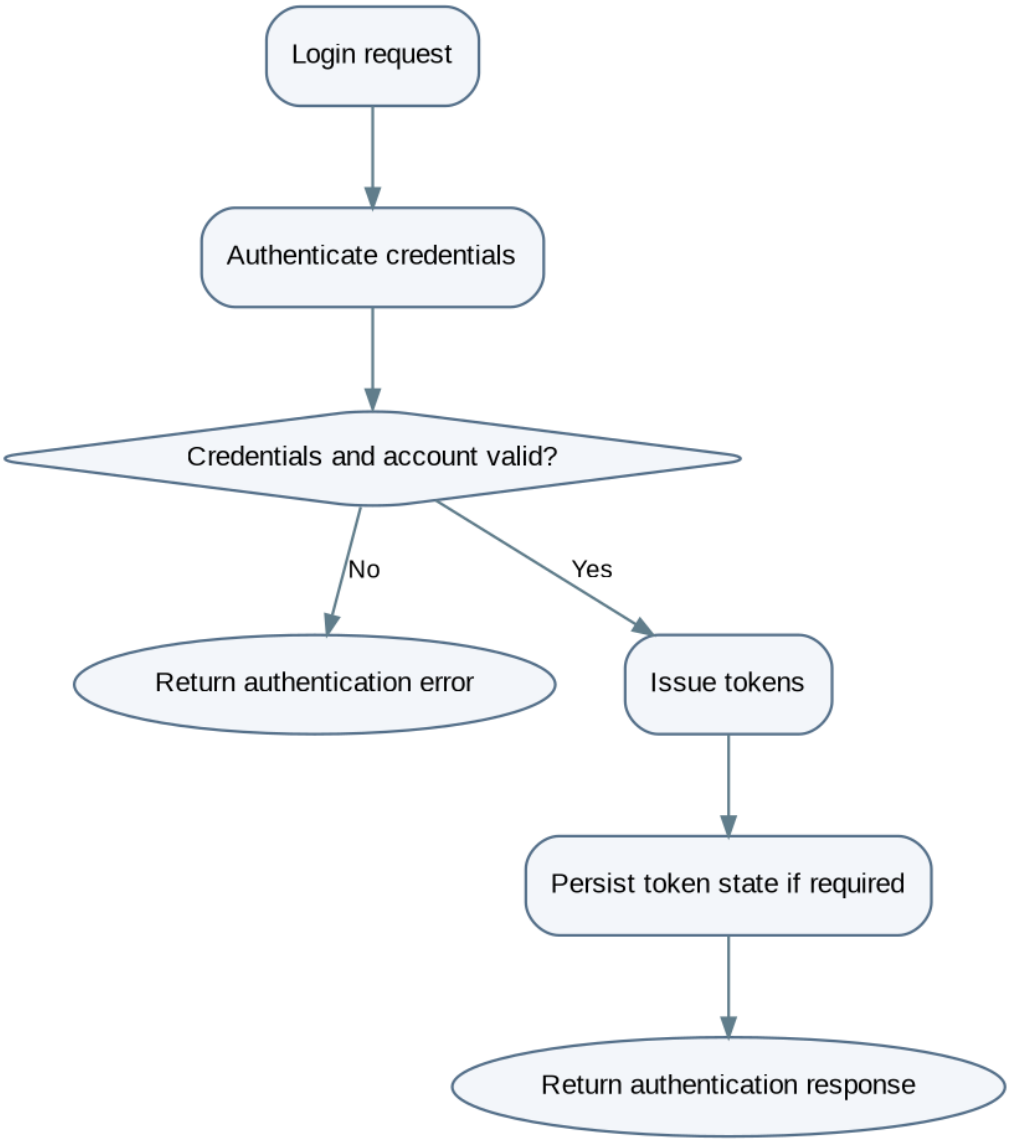


Figure 5. Authentication flow

Aspect	Detail
Data involved	`users`, `refresh_tokens`, `revoked_tokens`, `password_reset_tokens`

7.6.2 Administration and user management

Main path: create a user

Step	Action
1	An authorised administrator submits user registration data.
2	The API validates the request DTO.
3	The service checks required uniqueness and role/account rules.
4	The service creates the user account and associates any required employee profile.
5	Related records are saved within the implemented transaction boundary.
6	The response returns the created record or a standard success wrapper.

Other paths

Path	Flow
Role update	Authorise caller → validate target user and requested role → update role → return result.
Account status change	Authorise caller → validate target and new status → persist status → return result.
User / attendance lookup	Authorise caller → apply supported filters → load requested records → map to response DTOs.
Employee profile multipart creation/update	Validate text fields and photo metadata → process the file according to current storage logic → persist profile/file reference → return result.

> **Key checks** Only permitted roles should register users, assign privileged roles, or change another user's status. The exact policy is controlled by controller guards and service validation.

7.6.3 Employee management

Main path: create or update an employee profile

Step	Action
1	Receive employee details.
2	Validate required fields and referenced user account.
3	Resolve department, designation, and job title.
4	Validate relationships between the selected master records where implemented.
5	Resolve optional reporting manager.
6	Save or update the employee.
7	Return the mapped employee response.

Aspect	Detail
Profile-photo path	Receive multipart image → validate accepted type/size according to implementation → store the photo or its database reference → set/update photo metadata → return the response. Retrieval resolves the employee and returns the stored image/binary content.
Important relationships	• `employees.user_id` is unique. • Department, designation, and job-title references are required in the documented schema. • `reporting_manager_id` is an optional self-reference.

7.6.4 Organisation master data: departments, designations, and job titles

Create / update workflow

Step	Action
1	An authorised user submits a master-data request.
2	Validate required names, codes, and fields.
3	For a designation, resolve its department.
4	For a job title, resolve its designation.
5	Check uniqueness constraints and parent-child consistency.
6	Save the record and return the mapped response.

Aspect	Detail
Status / deletion workflow	Resolve record → check authorisation and any dependency rules → update active status or attempt deletion → return success or a domain/database error.

Open decision The schema summary establishes the hierarchy but does not establish whether deletion is blocked, soft-deleted, or permitted when employees already reference a record. Document the actual behavior from the service and migration constraints.

7.6.5 Lookup APIs

Step	Action
1	Client requests a dropdown or lookup list.
2	The controller receives any parent ID, such as department or designation.
3	The service queries the relevant master records.
4	The API returns IDs and display names for frontend selection.

Aspect	Detail
Documented examples	• Departments • Designations for a department • Job titles for a designation • Reporting managers • Employee ID / name pairs
Requirement	Lookup APIs should return only the fields needed by the caller and must follow the implemented access rules.

7.6.6 Attendance

Main path: daily attendance

Step	Action
1	Employee calls check-in.
2	Service resolves the employee, current date, attendance record, and active attendance settings.
3	It checks whether attendance is permitted for that day and whether a check-in already exists.
4	If valid, it records the check-in timestamp and applicable location data.
5	Break-start and break-end calls validate the current attendance state and record break intervals.
6	Checkout validates the current state and records checkout time.
7	Service calculates working time, break duration, late minutes, early departure, overtime, and attendance status according to the active settings.
8	The attendance record is saved and the API returns the result.
9	Read APIs provide today, history, monthly summary, calendar, and dashboard views from stored records and supported calculations.

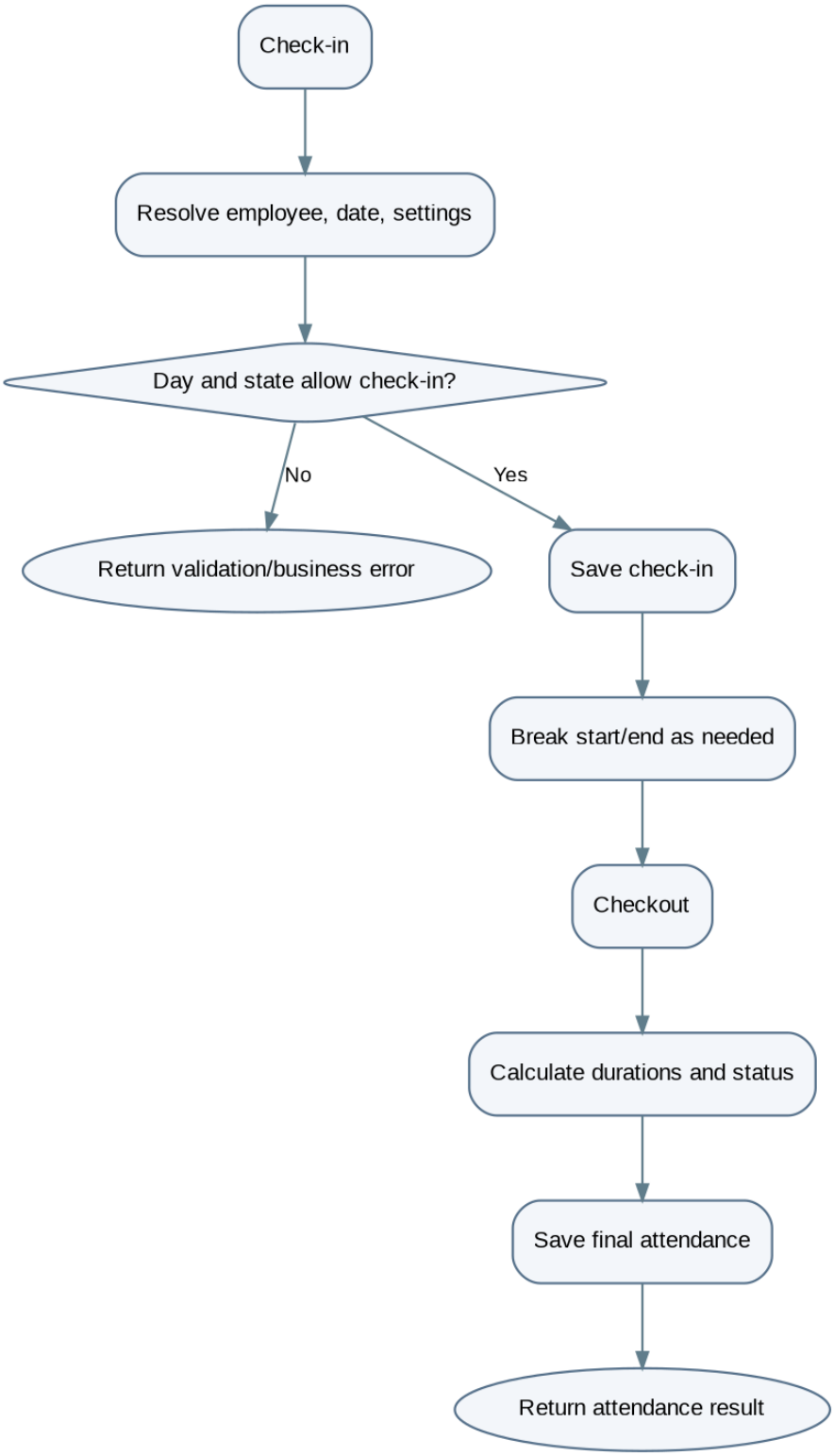


Figure 6. Daily attendance flow

Policy inputs (intended values)

Parameter	Intended value
Office start	9:30 AM
Office end	6:30 PM
Grace period	45 minutes
Present	8 hours
Half Day	4 hours
Lunch break	1:00 PM – 2:00 PM
Missed-checkout allowance	Up to 3 hours after the checkout time

> **Verify** These are intended policy values from earlier project specification. Verify the active `attendance_settings` row, configuration defaults, and calculation code before treating them as current runtime behavior.

Important edge cases

State-related	Rule / boundary-related
Duplicate check-in	Missed checkout
Checkout without check-in	Holiday / weekend attendance restrictions
Repeated checkout	Timezone / date boundaries
Open break at checkout	Concurrent requests

7.6.7 Attendance regularisation

Main path

Step	Action
1	Employee submits a regularisation request with a date range and correction details.
2	Service validates the range, ownership, required details, and existing attendance records.
3	The request and detail rows are saved in a pending state.
4	An authorised reviewer loads the request and chooses an allowed action for a detail.
5	The service records the reviewer, decision, timestamps, and remarks.
6	For an approved correction, the relevant attendance fields are updated according to the implementation.
7	A revert operation restores or reverts an approved detail according to the implemented rules.
8	The API returns the updated request/detail state.

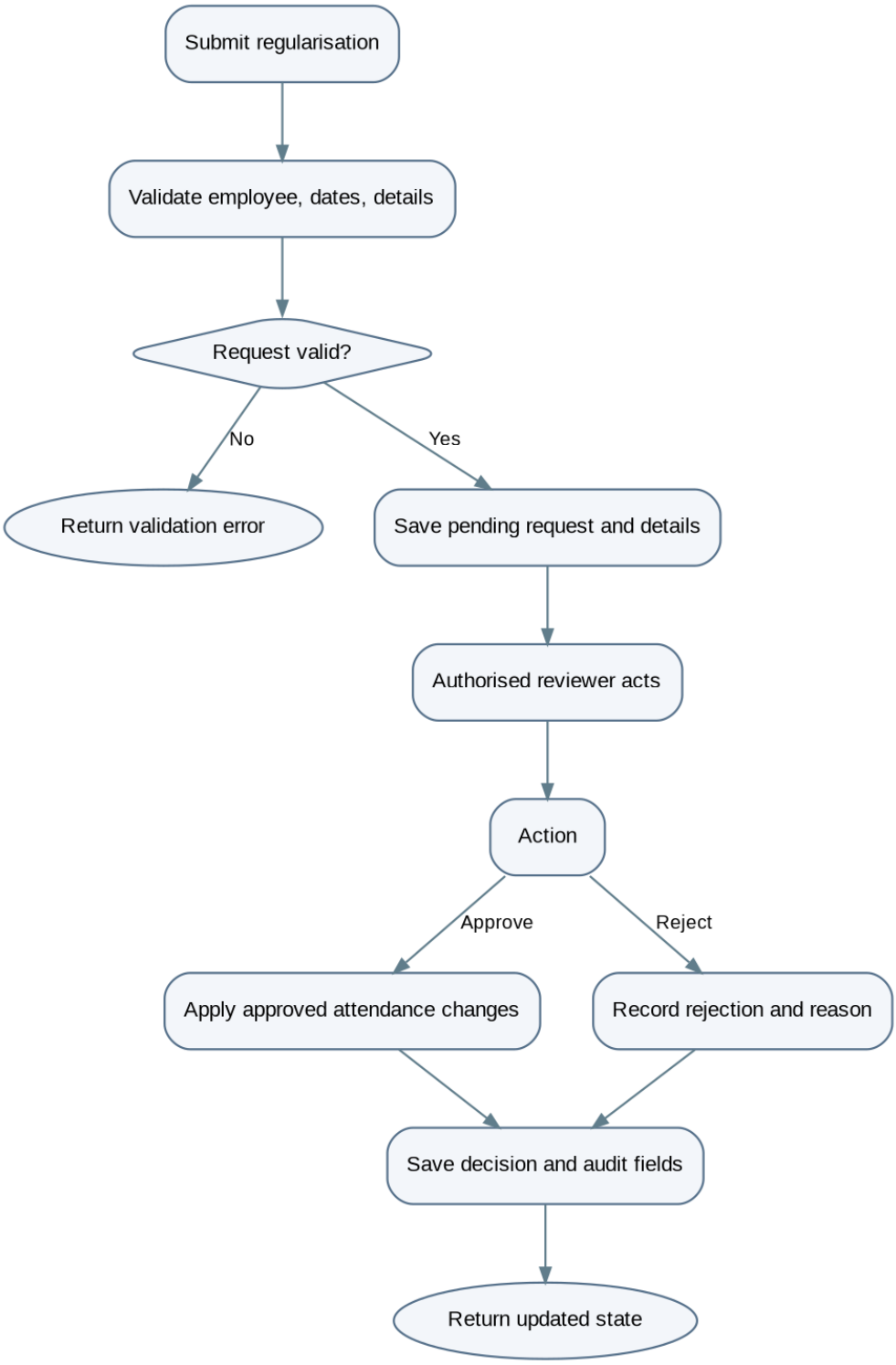


Figure 7. Attendance regularisation flow

7.6.8 Leave types and leave settings

Workflow	Flow
Leave-type maintenance	Authorised caller submits a create/update/status request → validate name and policy fields → enforce uniqueness and constraints → persist the leave type → return the updated record.
Leave-settings maintenance	Authorised caller reads or updates organisation leave settings → validate numeric limits and policy flags → persist annual/monthly guidance, carry-forward, and sandwich policy options → return settings.

> **Verify** These settings influence request validation and allocation. Updating a setting does not necessarily retroactively recalculate existing approved requests; confirm this in the service code.

7.6.9 Leave request application and validation

Main path: submit leave

Step	Action
1	Employee submits leave type, start/end dates, and reason.
2	Service resolves employee and active leave type.
3	It validates date ordering and required fields.
4	It checks overlap with pending or approved leave requests.
5	It calculates working days using the applicable weekend and holiday rules.
6	It applies configured sandwich rules and records any forced dates if supported.
7	It allocates working days by calendar month when required.
8	It creates the leave request in pending status.
9	It returns the request details.

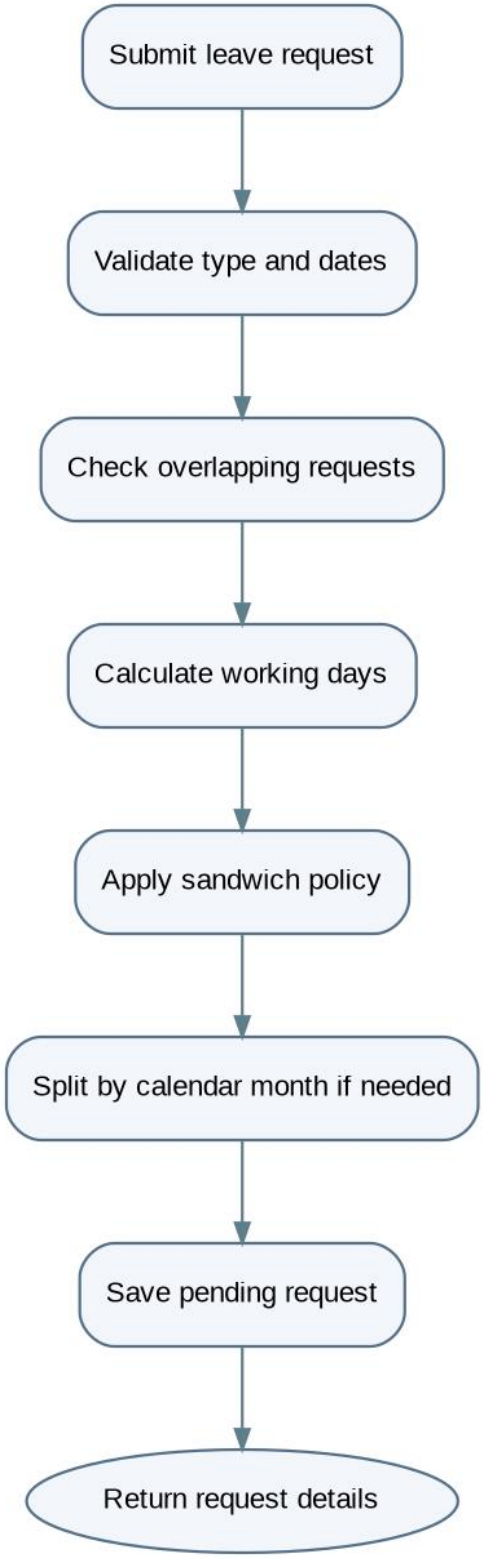


Figure 8. Leave submission flow

Sandwich-policy examples (intended policy)

Leave taken	Days counted	Chargeable days
Friday only	Friday + Saturday + Sunday	3
Monday only	Saturday + Sunday + Monday	3
Friday + Monday	Friday through Monday	4

> **Verify** These examples describe the intended policy and must be checked against `LeaveValidationServiceImpl` and the current `leave_settings` values.

7.6.10 Leave approval and rejection

Approval path

Step	Action
1	Manager / authorised approver requests pending leave items.
2	Service verifies the caller's role and relationship to the employee, where required.
3	Approver selects approve or reject and may provide remarks.
4	Service verifies that the request is still in a valid pending state.
5	On rejection, it records the decision and approval history without deducting paid leave balance.
6	On approval, it calculates paid days versus LOP for the applicable month(s).
7	It updates the request, records approver information, writes balance transactions and monthly allocations, and triggers configured notifications.
8	It returns the final decision state.

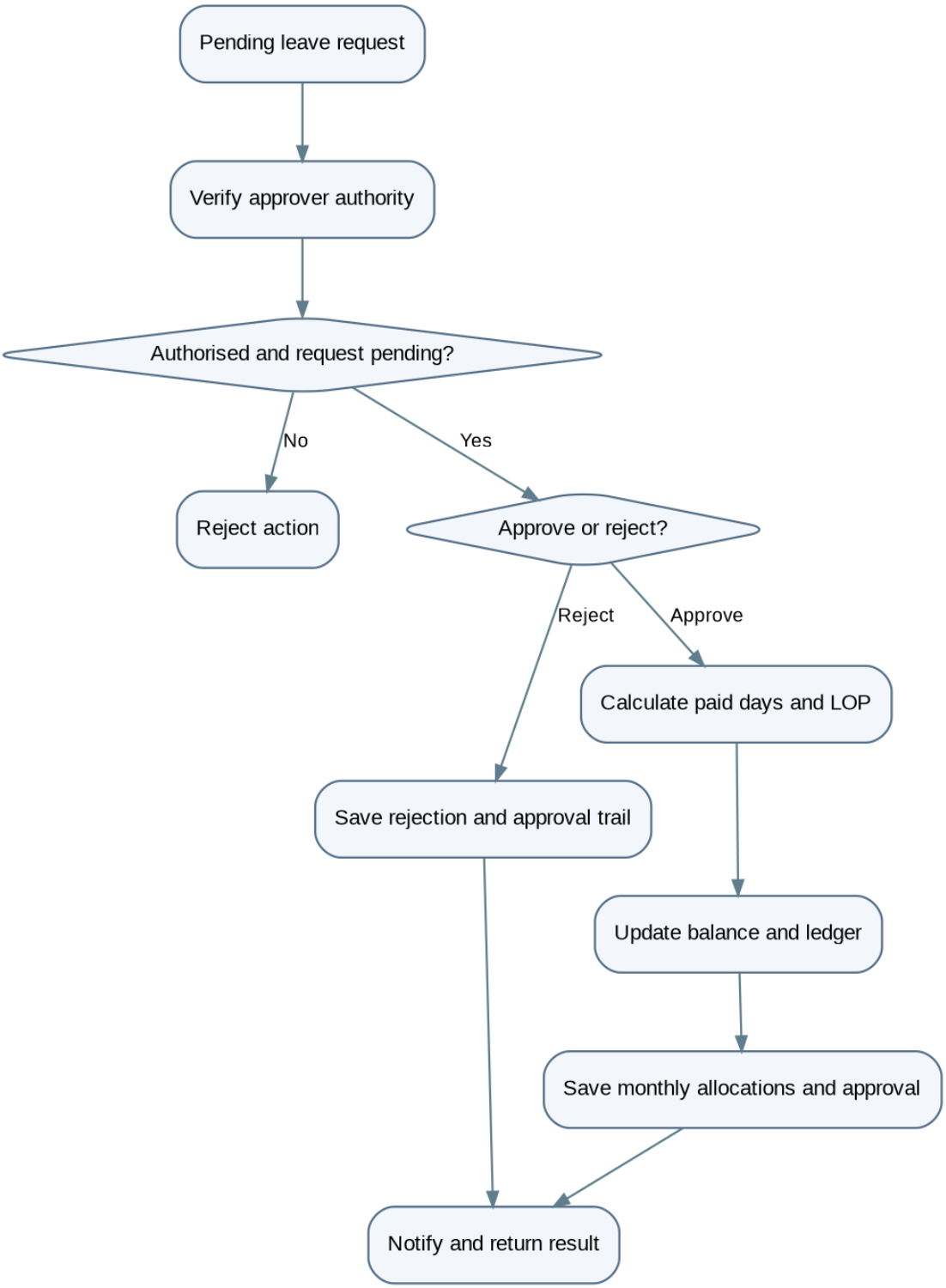


Figure 9. Leave approval flow

Rule	Detail
Paid / LOP rule	Paid days affect leave balance while LOP days do not deduct annual leave balance. The project policy previously specified up to two paid leaves per month, with additional approved leave treated as LOP. Verify that the current implementation applies the limit by calendar month and handles requests crossing month boundaries as intended.
Cancellation rule	The project requirement previously specified that approved leave should not be cancellable. Document the cancel endpoint as allowing cancellation only for statuses permitted by the approved behavior; do not describe approved-leave cancellation or balance restoration as supported unless the implementation proves it.

7.6.11 Leave balances, transactions, and allocation

Workflow	Flow
Balance lookup	Authenticate/authorise caller → resolve employee and year/type filters → read balance rows → return allocated, used, expired, and remaining values.
Transaction history	Resolve caller and requested employee/request → check access → query leave transactions → return ledger entries.
Allocation workflow	Authorised HR/admin invokes an allocation operation → service resolves eligible employees and leave type → validates allocation parameters and existing balances → creates or updates balance records and transaction history according to implementation → returns the allocation result.
Data consistency requirement	Whenever approval changes a balance, the balance change and corresponding transaction/allocation records should remain consistent. The actual transaction boundary and duplicate-allocation protection must be confirmed in the service and database constraints.

7.6.12 Holidays and calendar

Workflow	Flow
Holiday creation / update	Authorised user submits holiday details → validate date, name, type, recurring/active flags, and attendance permission → enforce unique holiday date → save → return the holiday.
Calendar / read workflow	Request calendar or upcoming holidays → service queries relevant dates and active records → combines data as implemented → returns calendar entries.

> **Open decision** Holiday records can affect attendance eligibility and leave working-day calculations. Confirm whether recurring holidays are expanded at query time or materialised as dated records; the supplied summary does not establish that detail.

7.6.13 Attendance, leave, and company settings

Settings update workflow

Step	Action
1	Authorised caller requests the current settings or submits an update.
2	Controller validates the request DTO.
3	Service validates time ranges, numeric limits, working days, company code, and policy flags as applicable.
4	Service saves the settings record.

Step	Action
5	Subsequent attendance, leave, or calendar operations read the applicable active settings.
6	API returns the saved configuration.

Effect of each settings area

Settings area	Effect
Attendance settings	Affect daily attendance and time calculations.
Leave settings	Affect allocation, carry-forward, and sandwich rules.
Company settings	Affect company identity and other configured organisation values.

7.6.14 Payroll setup: salary templates and salary structures

Workflow	Flow
Salary template workflow	Authorised payroll user creates or updates a template → validate salary component values and employee-type input according to current schema rules → persist template → return template.
Salary structure workflow	Select employee and optional template → validate employee/template and effective dates → calculate or map component values → create or revise the employee's structure → manage active/inactive state → return the structure.
Payment details workflow	Authorised caller submits employee payment and statutory details → validate required bank/account/IFSC and applicable payment mode → persist the employee's payment-details row → return a safe response.

> **Security** Sensitive account values should not be unnecessarily exposed in responses or logs.

7.6.15 Payroll generation, approval, payment, and payslip

Generation path

Step	Action
1	Payroll user selects employee/month or invokes the applicable generation operation.
2	Service validates employee and month.
3	It prevents a duplicate active payroll for the same employee/month.
4	It resolves the applicable salary structure.
5	It reads attendance and leave/LOP data for the period.
6	It calculates gross pay, LOP adjustment, half-day amount, deductions, and net payable.
7	It persists the payroll record and history.
8	It returns the generated payroll.

Other paths

Path	Flow
Lifecycle	A permitted status action is submitted → validate the current and requested statuses → validate role and transition rules → update status and relevant metadata → append `payroll_history` → return the updated payroll.
Regeneration	Load payroll → verify regeneration is permitted → reload applicable salary and attendance/leave inputs → recalculate derived values → save revised version/history according to implementation.
Payslip	Resolve payroll by ID → verify access and eligibility → generate PDF from the persisted payroll values → return PDF bytes.

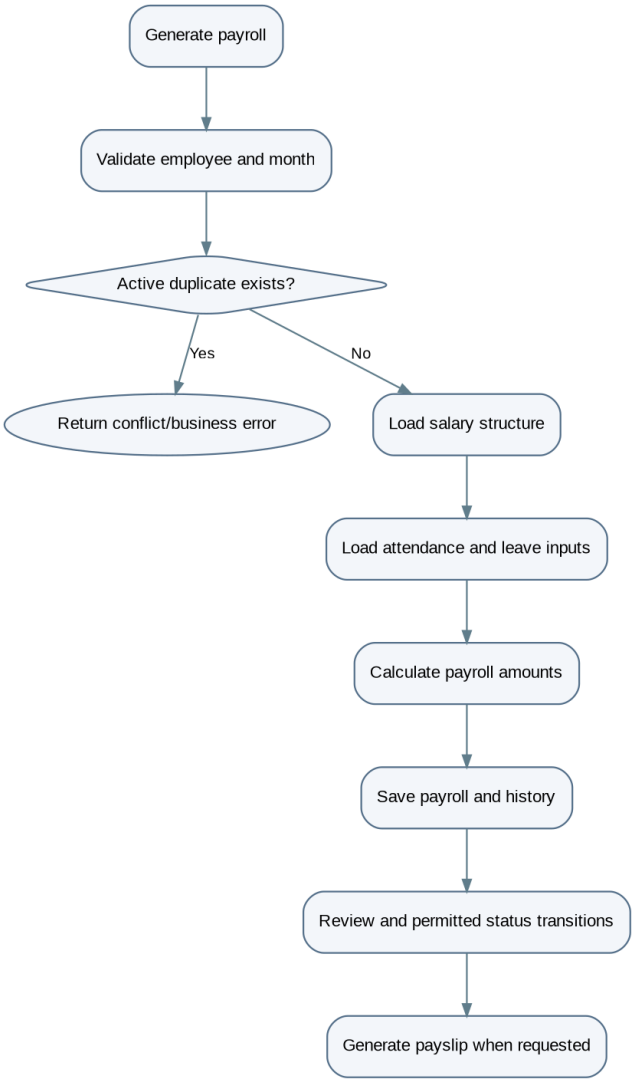


Figure 10. Payroll generation flow

Open decision Payroll calculation formulas and statutory deductions must be explicitly defined in the approved payroll rules and implemented consistently.

7.6.16 Notifications

Workflow	Flow
Read workflow	Authenticated user requests notifications → service scopes results to the permitted employee/user → applies paging/filtering → returns notifications and unread count.
Mark-read workflow	User selects one notification or all notifications → service verifies ownership/scope → updates read state → returns the updated result.
Creation triggers	Notifications may be created by other workflows, such as leave actions or announcements. The precise trigger list should be derived from calls to the notification service rather than assumed.

7.6.17 Announcements and attachment storage

Creation workflow

Step	Action
1	Authorised user submits announcement fields and multipart attachments.
2	Service validates announcement data and supported upload rules.
3	Attachments are uploaded to the configured S3-compatible storage.
4	The announcement record is saved.
5	Attachment URL rows are associated with the announcement.
6	The API returns the created announcement.

Workflow	Flow
Read workflow	Request announcement data → service retrieves announcement and attachment URL records → return the permitted response.

> **Verify** The exact ordering and cleanup strategy for failures between object upload and database commit must be confirmed in the implementation. Avoid claiming atomicity across database and object storage unless compensating logic exists.

7.6.18 Celebration wall and birthday information

Aspect	Detail
Workflow	Request today's celebration or upcoming birthdays → service determines the applicable date range/timezone → queries eligible employee data and relevant announcements → applies visibility rules → returns celebration/birthday entries.
Gaps	The endpoint register confirms these read operations but does not fully specify birthday date handling, privacy rules, or timezone behavior. Those details require source verification.

7.6.19 Reports and exports

Step	Action
1	Caller requests an attendance or leave report with supported filters.
2	Controller validates filter inputs and access.

Step	Action
3	Service constructs the appropriate query.
4	Repository retrieves matching records and aggregate values.
5	Service maps results into report DTOs.
6	For an export request, the relevant exporter generates the supported output format.
7	The API returns JSON or file/binary content as implemented.

Verify Verify the exact supported formats, filters, aggregation formulas, and role restrictions from each report controller and service. Do not assume every report supports every export format.

7.6.20 Performance reviews

Workflow	Flow
Create	Authorised reviewer submits review data → service validates employee/reviewer, period, type, rating/score and required fields → checks for a duplicate employee/type/month/year record → persists review → returns result.
Update	Load review → verify access and editable status → validate new values → save and return.
Complete	Load review → verify permitted transition and required fields → change review status to completed → save and return.
Search / summary	Validate filters → query review records → aggregate or map results → return list or summary.
Delete	Resolve review → authorise caller → check deletion rules → delete or deactivate as implemented → return result.

Note The unique constraint is an important final guard against duplicate period reviews, but the exact rating scale and completion prerequisites must be confirmed from the implementation.

7.6.21 Form 16

General workflow (based on the documented module structure)

Step	Action
1	Resolve the employee and create or retrieve the applicable Form 16 record.
2	Verify that the caller has permission to create or read the record.
3	Maintain related salary, exemption, Chapter VI-A, section 16 deduction, and final-field data through the applicable endpoints.
4	Maintain employer master information where required.
5	Maintain quarter, challan, and verification information through the associated APIs.
6	Return the relevant Form 16 record or subsection response.

Open decision The exact parent-child relationships, required subsection order, calculation behavior, and finalisation rules cannot be confirmed from the supplied summary alone. They must be derived from the Form 16 entities, DTOs, services, controllers, and Flyway migration V5.

7.6.22 Form 16A

Aspect	Detail
Required coverage	Record creation, validation, updates, retrieval, and applicable export or finalisation actions as defined by the approved tax requirements.
How to complete	Trace the Form 16A controller entry point → request DTO validation → service method → entity/repository operations → response mapping. Add its specific steps only after inspecting those files.
Caution	Do not assume that Form 16A has the same lifecycle or relationships as Form 16.

7.6.23 Cross-module workflow: employee setup to HR operations

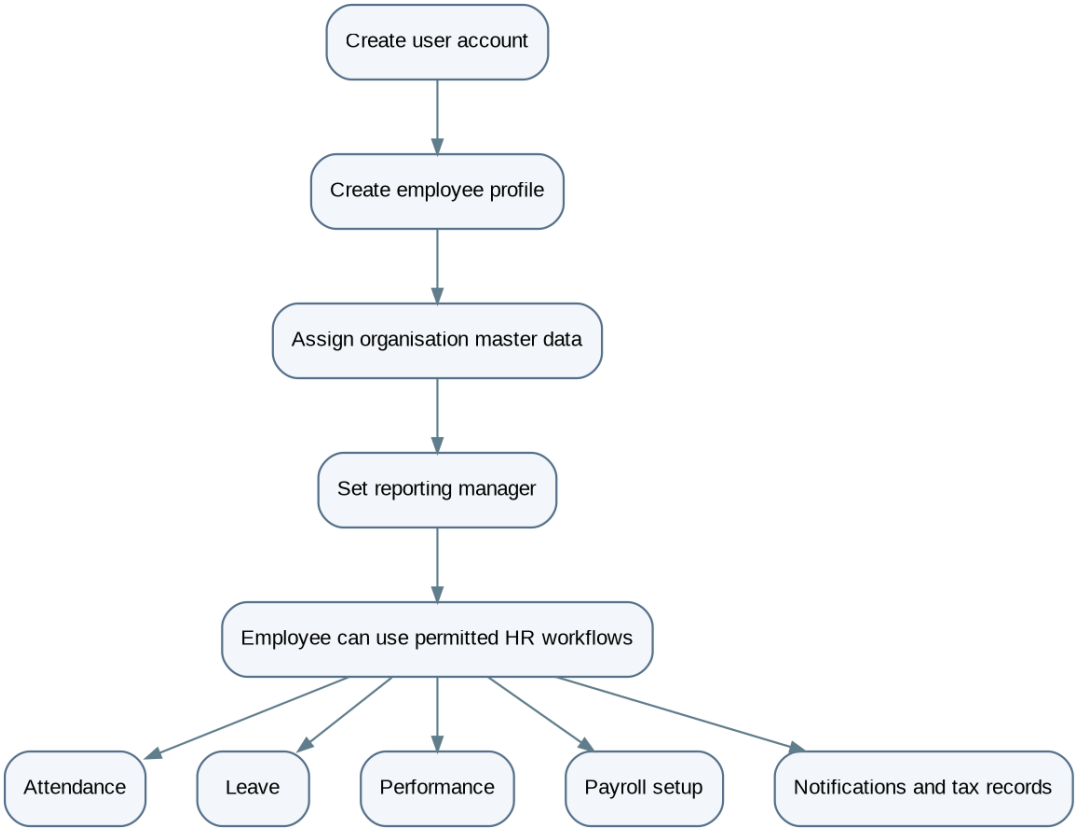


Figure 11. Employee setup to HR operations

Note This is a logical dependency flow, not necessarily a single transaction. Each operation has its own API, validation, and persistence behavior.

7.6.24 Cross-module workflow: attendance and leave to payroll

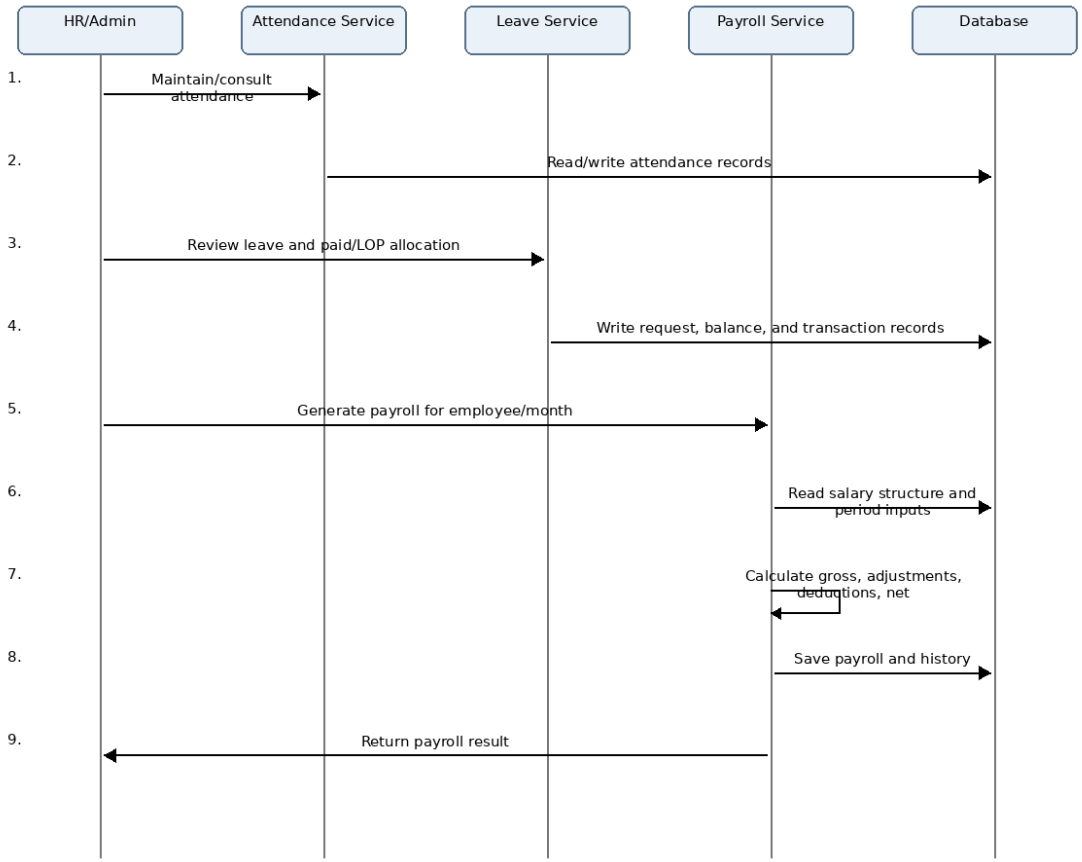


Figure 12. Attendance and leave to payroll

Verify The exact period cutoffs and calculation inputs must be confirmed in `PayrollServiceImpl`.

8. Security requirements

Control	Requirement
Session model	`SecurityConfig` uses stateless Spring Security; CSRF is disabled; CORS is enabled.
Authentication	Authenticates through DAO user details / password encoder; the JWT filter is added before the username/password filter.
Error handling	Unauthenticated and forbidden exceptions are sent to dedicated handlers.
Public patterns	Auth, Swagger/OpenAPI and health. All other requests require authentication.
Method security	Method-level `@PreAuthorize` is enforced because `@EnableMethodSecurity` is enabled.
Input validation	DTO Bean Validation plus service business validation.
Client trust	Do not rely on a client role/menu for authorisation.

9. Implementation roadmap

The following implementation sequence groups work by dependency and functional readiness.

Phase	Deliverables	Acceptance criterion
1 Foundation	PostgreSQL/Flyway, base audit/error/response, security	Clean schema, authenticated protected request, standard failure response.
2 Organisation	Roles/users, masters, employees/lookups	Hierarchy and reporting manager persist with constraints.
3 Time and leave	Settings/holidays, attendance, regularisation, leave/balance/approvals	End-to-end check-in and paid/LOP/sandwich approval correctness.
4 Compensation	Payment, templates, structures, payroll/history/payslip	Reproducible calculations and duplicate-month prevention.
5 HR operations	Notifications, reports, performance, Form 16	Guarded CRUD/export records and scheduled side effects.
6 Hardening	API contract review, security and data-integrity verification	Every active controller route has accepted/denied/invalid-input evidence.

Open design items Finalize exact request and response fields, authorization rules, validation messages, and any behavior not explicitly defined in the requirements before implementation.

10. Detailed module guide

Use this section as a module reference. Each module description covers its purpose, main data, key rules, and dependencies.

10.1 Authentication and account lifecycle

Aspect	Detail
Objective	Provide a controlled entry point to the backend and manage the lifecycle of user credentials and tokens.
Required behavior	1. Accept login credentials and authenticate them through Spring Security's authentication mechanism. 2. Return access and refresh tokens after successful authentication. 3. Validate refresh tokens before issuing replacement access credentials. 4. Revoke refresh-token state during logout. 5. Support authenticated password change and a reset flow based on expiring, hashed reset tokens. 6. Expose the current authenticated user's profile. 7. Reject invalid credentials, expired or revoked tokens, and accounts that cannot authenticate.
Primary components	AuthenticationController, AuthenticationServiceImpl, Spring Security configuration, user-details implementation, and refresh/revoked/password-reset token entities.

Aspect	Detail
Data involved	`users`, `refresh_tokens`, `revoked_tokens`, `password_reset_tokens`
Design considerations	Token expiry, revocation, account status, password-change time, and reset-token reuse are security-sensitive state. A reset token should be accepted only when its hash matches, it has not expired, and it has not already been used. Confirm these conditions in the service implementation rather than inferring them from the existence of token entities.

10.2 Administration and employee management

Aspect	Detail
Objective	Allow authorised administrators to provision accounts, assign roles, manage employee profiles, and perform permitted administrative reviews.
Main operations	• Register a user and establish the corresponding employee profile where required. • Assign or change a user's role. • Activate, deactivate, or otherwise change account status. • Create and update employee profile data. • Upload and retrieve profile photos. • Retrieve employee lists, individual profiles, and employee dropdown data. • Resolve department, designation, job title, and reporting-manager references.
Data involved	`users`, `employees`, `departments`, `designations`, `job_titles`, and related lookup data.
Integrity requirements	• An employee's `user_id` must refer to one user and is unique. • Department, designation, and job-title references must point to valid master records. • `reporting_manager_id` is an optional self-reference to another employee. • Employee code uniqueness is introduced by migration V3. • Profile updates should not allow a caller to assign arbitrary privileged roles or modify another employee's protected data without the required permission.
Verification note	The last integrity point is a security requirement to verify against the actual authorization and service code; it is not proof that every update path currently enforces it.

10.3 Organisation master data

Aspect	Detail
Objective	Maintain the organisation hierarchy used by employee profiles and downstream HR processes.
Hierarchy	**Department → Designation → Job Title → Employee assignment**
Relationship basis	A designation references a department and a job title references a designation. Employees reference department, designation and job title, which lets the application present human-readable master names while storing relational IDs.

Aspect	Detail
Expected validation points	• A referenced master record exists. • Inactive master records are handled consistently when assigning or editing employees. • A designation belongs to the department selected for the employee. • A job title belongs to the selected designation. • Deletion or deactivation does not leave invalid employee references.
Open decision	Deletion and cross-level consistency rules must be defined for organisation master records, particularly when employees or other records reference them.

10.4 Attendance

Aspect	Detail
Objective	Record daily attendance and calculate attendance metrics using configurable company rules.
Main operations	Check-in, break start, break end, checkout, today's attendance, history, monthly summary, calendar view, and dashboard summary.
Relevant data	`attendance`, `attendance_breaks`, `attendance_settings`, `holidays`, `employees`
Status vocabulary	`PRESENT`, `LATE`, `HALF_DAY`, `ABSENT`, `LEAVE`, `HOLIDAY`, `WEEKEND`, `MISSED_CHECKOUT`
Processing requirements	1. Resolve the authenticated employee and the relevant attendance date. 2. Load active attendance settings and relevant holiday/weekend rules. 3. Validate whether the requested transition is allowed from the current attendance state. 4. Record check-in or break timestamps as applicable. 5. On checkout, calculate work duration and related metrics from recorded timestamps and breaks. 6. Derive lateness, early departure, overtime, and attendance status using the configured settings. 7. Persist the result and return the appropriate response. 8. Reject invalid transitions, duplicate actions, or operations disallowed by company settings.
Verification note	The supplied summary confirms the feature set and status values, but does not contain the exact formulas or every transition condition. Check AttendanceServiceImpl and related calculation methods before publishing these as exact policy.

10.5 Attendance regularisation

Aspect	Detail
Objective	Allow eligible employees to request correction of attendance records and allow authorised reviewers to act on individual correction details.
Data involved	`attendance_regularization`, `attendance_regularization_detail`, `attendance`, `employees`
Documented lifecycle	A request is submitted for a date range; details represent the individual attendance corrections; each detail can be approved, rejected, or reverted according to its status rules.

Aspect	Detail
Workflow requirements	• Validate that the employee may request regularisation for the selected dates. • Prevent invalid date ranges and malformed requested timestamps. • Preserve original attendance values alongside requested and approved values. • Record who approved or rejected the correction and any reason. • Apply approved corrections to attendance data. • Restrict action and revert operations to authorised roles. • Preserve an auditable distinction between the original record, requested change, approved change, and current state.
Open decision	Exact rules for allowed historical dates, reviewer hierarchy, and whether one invalid detail rejects the entire request require source verification.

10.6 Leave management

Aspect	Detail
Objective	Manage leave applications, approvals, leave balances, paid leave, loss of pay (LOP), monthly allocation, and an auditable transaction history.
Main data structures	• `leave_types` — configurable leave category and policy properties. • `leave_settings` — organisation-level annual/monthly and sandwich policy settings. • `leave_requests` — employee request, date range, status, paid/LOP values, approver, and forced-date information. • `leave_approvals` — approval/rejection action history. • `leave_balances` — employee/type/year balance. • `leave_transactions` — auditable balance movements. • `leave_request_month_allocations` — per-month allocation for requests spanning calendar months.
Request workflow requirements	1. Validate the leave type and ensure it is active. 2. Validate the date range and request input. 3. Detect overlap with existing pending or approved requests. 4. Calculate working days using weekend and holiday rules. 5. Apply configured sandwich policy where applicable. 6. Save a pending request and its month allocation information when required. 7. Authorise the approver against the employee reporting hierarchy and applicable role. 8. On rejection, record the decision without applying an approval balance deduction. 9. On approval, classify eligible days as paid or LOP using the applicable monthly and balance rules. 10. Write corresponding balance updates, transaction records, approval history, and request status consistently. 11. Trigger the documented notification/email side effects where configured.
Paid versus LOP	Approved paid days affect leave balance and leave transactions, while LOP days do not deduct annual balance. V9 introduces paid/LOP/approver fields, V10 adds monthly allocations, and V11 adds sandwich settings and forced-date storage — the model evolved to support more explicit allocation logic.

Aspect	Detail
Sandwich leave	Documented settings include Monday leave, Friday leave, and Friday-plus-Monday policy flags. Weekend dates may be forced into the leave calculation. The exact rules for each combination must be verified in `LeaveValidationServiceImpl`; the settings alone do not fully define every date-calculation edge case.
Cancellation	The existing workflow summary includes owner cancellation and applicable balance restoration. Because cancellation behavior can vary by request status, check whether pending and approved requests are treated differently. Do not assume approved requests can be cancelled merely because a cancellation endpoint exists.

10.7 Leave approvals and allocation

Aspect	Detail
Objective	Separate approval decisions from leave requests and support allocation of leave to employees.
Data	`leave_approvals` records the request, approver, action, approval level, and remarks. `leave_request_month_allocations` stores monthly paid/working-day attribution for a request.
Why separate	The separation provides an audit trail and allows leave calculations to be attributed to individual calendar months. For a request crossing a month boundary, the service should calculate the allocation for each relevant month rather than treating the whole request as belonging to one month.
Open decision	The supplied summary does not list every allocation API parameter or every allocation algorithm branch; those details require inspection of the allocation request DTO and service implementation.

10.8 Payroll and compensation

Aspect	Detail
Objective	Maintain salary configuration and generate payroll records with traceable lifecycle changes and payslip output.
Main data structures	• `salary_templates` — reusable salary component configuration. • `salary_structures` — employee-specific salary configuration with effective dates and status. • `employee_payment_details` — employee bank and statutory payment information. • `payrolls` — calculated employee/month payroll record. • `payroll_history` — audit trail of payroll actions.
Payroll lifecycle	`DRAFT` → `GENERATED` → `APPROVED` → `PAID`; also `SUPERSEDED`, `CANCELLED`.
Generation requirements	1. Resolve the employee and target payroll month. 2. Check for an existing active payroll for the same employee/month. 3. Resolve the applicable active salary structure. 4. Read relevant attendance and leave/LOP information. 5. Calculate gross pay, LOP amount, half-day amount, deductions, and net payable. 6. Persist the payroll record and corresponding history entry. 7. Return the generated payroll representation.

Aspect	Detail
Change & regeneration	When payroll data is updated or regenerated, calculated values should be recomputed consistently. Lifecycle transitions should be recorded in `payroll_history`. Payslip generation should use the persisted payroll record and return a PDF/binary response.
Schema evolution	V6 adds half-day count and amount fields. V7 removes the original employee-type check and unique restriction from salary templates, so the current rule must be derived from the later migration state, not the baseline alone.
Open decision	Payroll calculation formulas, salary components, statutory deductions, and rounding rules must be explicitly documented in the approved payroll specification.

10.9 Holidays, calendar, and settings

Aspect	Detail
Objective	Centralise configuration that affects attendance, leave, and company-level HR behavior.
Stored configuration	• `holidays` — date, name, description, active/recurring flags, attendance permission, and holiday type. • `attendance_settings` — office timing, grace/cutoff values, minimum attendance durations, overtime behavior, and weekend/holiday attendance flags. • `leave_settings` — leave allocation and sandwich policy settings. • `company_settings` — company identity, contact/address, working days, timezone, currency, and company code.
Why it matters	These settings are inputs to business logic rather than display preferences. Changes can affect date calculations and attendance/leave decisions, so the service layer should validate values and handle changes consistently.
Verification note	Exact constraints and update permissions should be confirmed in the respective request DTOs and controllers.

10.10 Notifications, announcements, and celebrations

Aspect	Detail
Objective	Deliver employee-facing notifications and maintain announcements with attachment references.
Notification data	`notifications` records employee ownership, message/title, type, priority, read status, and optional reference metadata.
API capabilities	• Listing and unread count • Marking one notification read / marking all read • Today's announcement • Upcoming birthdays and celebration-wall data • Multipart announcement creation
Attachment storage	Announcement attachment URLs are stored in `announcement_attachment_urls`, linked to announcements. Files are uploaded through an S3-compatible Backblaze B2 service.

Aspect	Detail
Important checks	• Validate upload type and metadata. • Control who can create announcements. • Handle upload failure. • Avoid orphaned stored files if database persistence fails.
Open decision	The exact compensation/cleanup behavior for partial upload failures is not established by the supplied summary.

10.11 Reports

Aspect	Detail
Objective	Provide filtered views and exports for attendance and leave data.
Scope	The documented report endpoints cover attendance and leave. Filters should be described from the actual filter DTOs and query logic, including date ranges, employee filters, and pagination if present.
Data principle	Report results are derived views of authoritative attendance and leave records, not independent business records.
Verification note	The summary confirms JSON and Excel/PDF export behavior generally, but does not enumerate the precise supported format for each endpoint. That mapping requires controller inspection.

10.12 Performance management

Aspect	Detail
Objective	Maintain employee performance review records, including review period/type, status, ratings or scores, feedback, and summary views.
Data	`performance_reviews` references employees and reviewers and has a unique constraint on employee / type / month / year.
API support	Create, search, summary, update, completion, retrieval, and deletion.
Duplicate protection	The unique constraint prevents duplicate records for the same employee/review type/month/year at the database level. The application should also return a clear domain error when a duplicate is attempted.
Open decision	Exact score ranges, required feedback, and permitted reviewer relationships must be defined in DTO validation and service logic.

10.13 Form 16 and Form 16A

Aspect	Detail
Objective	Maintain Indian tax-related employee and employer records, including salary details, exemptions, deductions, Chapter VI-A values, section 16 deductions, verification, quarters, and challan information.
Current state	The document identifies Form 16 and Form 16A packages and says the Form 16 table suite is created or altered by V5. It does not enumerate the full physical schema or all relationships in sufficient detail to safely produce a complete Form 16 / Form 16A ERD subgraph.

Aspect	Detail
Before claiming completeness, inspect	• All entities under `form_16/entity` and `Form_16A/entity`. • The complete V5 migration and any later migrations affecting those tables. • Every `@ManyToOne`, `@OneToMany`, `@OneToOne`, and join-column mapping. • The Form 16 / Form 16A controller routes and request/response DTOs.

11. Database relationship reference

Parent / referenced record	Related record	Relationship and meaning
`users`	`employees`	One-to-one by unique `employees.user_id`; employee profile belongs to a user account.
`departments`	`designations`	One-to-many; a department can contain multiple designations.
`designations`	`job_titles`	One-to-many; a designation can have multiple job titles.
`employees`	`employees`	Self-referencing manager hierarchy through `reporting_manager_id`.
`employees`	`attendance`	One employee can have multiple attendance records over time.
`attendance`	`attendance_breaks`	One attendance record can have multiple break intervals.
`employees`	`attendance_regularization`	One employee can submit multiple regularisation requests.
`attendance_regularization`	`attendance_regularization_detail`	A request can contain multiple correction details.
`employees / leave_types`	`leave_balances`	Balances are maintained per employee, leave type, and year; the combination is unique.
`employees / leave_types`	`leave_requests`	Requests belong to an employee and a leave type.
`leave_requests`	`leave_approvals`	One request can have multiple approval trail entries.
`leave_requests`	`leave_transactions`	Transactions can refer to a leave request and record balance movements.
`leave_requests`	`leave_request_month_allocations`	One request can have monthly allocation rows; request/month is unique.
`employees`	`salary_structures`	Salary structures are associated with an employee.
`salary_templates`	`salary_structures`	A structure may reference a template; the template reference is optional in the documented schema.
`employees / salary_structures`	`payrolls`	Payroll records reference an employee and salary structure.

Parent / referenced record	Related record	Relationship and meaning
payrolls	payroll_history	A payroll can have multiple lifecycle history records.
employees	employee_payment_details	Unique employee reference allows at most one payment-details row per employee in the documented schema.
employees	performance_reviews	Employee reviews are associated with an employee; a reviewer is also referenced.
employees	notifications	Notifications are associated with a recipient employee.
announcements	announcement_attachment_urls	An announcement can have multiple attachment URL entries.
users	refresh_tokens / password_reset_tokens	A user can have multiple token records over time.

Tax-data design decision Confirm and include all Form 16 and Form 16A entities and their relationships in the final ERD before implementation sign-off.

12. API contract standards

The API register in §6 is a module-level endpoint inventory. For a truly exhaustive API contract, each controller method must be checked for its exact class-level and method-level mapping, parameters, DTO fields, response type, and annotations.

For every endpoint, capture the following record

Contract item	What to record
Route	Combined class-level and method-level mapping, including path variables.
Method	GET, POST, PUT, PATCH, DELETE, or other declared method.
Input	Query/path parameters, request DTO fields, multipart parts, and headers.
Validation	`@Valid`, field constraints, custom validators, and service-level checks.
Access	Class-level and method-level `@PreAuthorize`, plus global security rules.
Processing	Controller handler and service methods invoked.
Persistence	Repository calls and entities created, changed, or read.
Output	Actual return type, response wrapper, file/binary type, and status annotation.
Failure	Explicit exceptions and global exception mapping.

Note The route register should not be treated as exhaustive where it uses shorthand such as `GET /`, grouped resource names, or "CRUD variants." Replace those entries with exact mappings from the source before using this document as a definitive API contract.

13. How modules share data

13.1 Employee setup to operational use

Step	Action
1	An authorised administrator creates or manages the user account.
2	An employee profile references the user and relevant organisation master records.
3	Reporting-manager relationships establish the basis for manager-scoped operations.
4	The employee profile becomes the shared reference for attendance, leave, payroll, performance, notifications, and tax records.

13.2 Attendance and leave to payroll

Step	Action
1	Attendance stores time and daily status information.
2	Leave requests store approved leave periods and paid/LOP allocation.
3	Payroll generation reads the employee's salary structure and applicable attendance/leave information.
4	Payroll calculations produce gross, adjustments, deductions, and net payable.
5	Payroll history records lifecycle actions, while payslip generation presents the persisted result.

> **Verify** The summary establishes that attendance and leave feed payroll calculations. The precise query joins, date cutoff rules, and calculation formulas must be confirmed from `PayrollServiceImpl` and its collaborating services.

13.3 Settings to operational decisions

Input	Influences
Attendance settings	Attendance classification and duration calculations.
Holidays and weekend rules	Attendance and leave-day calculations.
Leave settings	Paid allocation, carry-forward configuration, and sandwich calculations.

These relationships make settings part of the business-rule execution path, not isolated configuration screens.

13.4 Requirement traceability

Requirement area	Related components	Data model / outcome
Authentication	AuthenticationController, AuthenticationServiceImpl, SecurityConfig	users, refresh/revoked/reset token tables
Employee and organisation	AdminServiceImpl, employee/master/lookup packages	employees, departments, designations, job_titles

Requirement area	Related components	Data model / outcome
Attendance	AttendanceServiceImpl, AttendanceSettings	attendance, attendance_breaks, attendance_settings, holidays
Regularisation	AttendanceRegularizationServiceImpl	Regularisation header/detail and attendance records
Leave	Leave validation, authorization, paid/LOP, balance, transaction, approval and allocation services	Leave requests, types, settings, balances, approvals, transactions, monthly allocations
Payroll	PayrollServiceImpl and PDF-related classes	Templates, structures, payment details, payroll, history
Notifications and announcements	Notification services / controllers	notifications, announcements, attachment URL table
Reports	report package	Derived attendance and leave responses/exports
Performance	PerformanceServiceImpl	Performance review records
Tax forms	form_16/**, Form_16A/**, V5 migration	Form 16 / Form 16A tables; full mapping requires source verification